

M2 File Format Specification

Format: Blizzard M2 Model Format (World of Warcraft)

Byte Order: Little-endian

Primary Reference: <https://wowdev.wiki/M2>

Implementation: WhiteoutLib `whiteout::m2` — parser + writer for `.m2`, `.skin`, `.anim`, `.bone`, `.skel`

Table of Contents

- M2 File Format Specification
 - Table of Contents
 - 1. Overview
 - 2. Format Variants
 - 2.1 Classic (MD20)
 - 2.2 Chunked (MD21)
 - 3. Version History
 - 4. Primitive Types
 - Scalar Types
 - Normalized Integers
 - Vector / Matrix Types
 - Compressed Quaternion (CompatQuaternion)
 - Extent (Bounding Box + Sphere)
 - 5. Core Patterns
 - 5.1 M2Array
 - 5.2 Animation Tracks
 - 5.3 Interpolation
 - 5.4 Global Sequences
 - 5.5 FBlock (Fake-AnimationBlock)
 - 6. MD20 Header
 - 6.1 Binary Layout
 - 6.2 Global Flags
 - 7. Skeleton and Animation
 - 7.1 Global Sequence Loops
 - 7.2 Animation Sequences
 - 7.3 Sequence Flags
 - 7.4 Sequence Index Hash (Animation Lookup)
 - 7.5 Bones
 - 7.6 Bone Flags
 - 7.7 Key-Bone Lookup
 - 8. Geometry and Rendering
 - 8.1 Vertices
 - 8.2 Textures
 - 8.3 Texture Types
 - 8.4 Materials (Render Flags)

- 8.5 Colors and Transparency
- 8.6 Texture Transforms
- 8.7 Lookup / Combo Tables
- 9. Bounding and Collision Volumes
- 10. Scene Objects
 - 10.1 Attachments
 - 10.2 Events
 - 10.3 Lights
 - 10.4 Cameras
- 11. Effects
 - 11.1 Ribbon Emitters
 - 11.2 Particle Emitters
 - 11.3 Particle Flags
- 12. SKIN File (.skin)
 - 12.1 Skin Profile
 - lodVertexBase — HD Model Vertex Splitting
 - 12.2 Skin Sections (Submeshes)
 - 12.3 Batches (Texture Units)
 - 12.4 Shadow Batches
 - 12.5 HD Model Identification
- 13. Chunked M2 (MD21 + Chunks)
 - 13.1 Chunk Format
 - 13.2 Chunk Catalogue
 - 13.3 File ID Reference Chunks
 - 13.4 Particle Extension Chunks
 - 13.5 Parent Data Chunks
 - 13.6 LOD and Rendering Chunks
 - 13.7 Waterfall / PBR Chunks
 - 13.8 Collision and Housing Chunks
 - 13.9 Miscellaneous Chunks
- 14. Skeleton File (.skel)
- 15. Bone File (.bone)
- 16. Animation File (.anim)
 - Pre-Legion (raw)
 - Legion+ (chunked)
- 17. Physics File (.phys) and PFDC
 - 17.1 File Structure Overview
 - 17.2 Version History
 - 17.3 PHYS Header
 - 17.4 PHYT — Physics Type
 - 17.5 Body Structures
 - 17.6 Shape Structures
 - 17.7 Shape Data: CAPS (Capsule)
 - 17.8 Shape Data: PLYT (Polytope) — CORRECTED
 - 17.9 Joint Structure
 - 17.10 Joint Types

- [SHOJ \(Shoulder Joint\)](#) — 116 bytes
 - [SHJ2 \(Shoulder Joint v2\)](#) — 124 bytes
 - [WLJ2 \(Weld Joint v2\)](#) — 112 bytes
 - [WLJ3 \(Weld Joint v3\)](#) — 116 bytes
 - [REVJ \(Revolute Joint\)](#) — 112 bytes
 - [REV2 \(Revolute Joint v2\)](#) — 120 bytes
- [17.11 PFDC Inline Physics](#)
- [18. Bundle Naming Conventions](#)
- [19. WhiteoutLib Implementation Notes](#)
 - [19.1 Parser](#)
 - [19.2 Writer](#)
 - [19.3 API Surface](#)
 - [19.4 Known Limitations](#)
- [20. Corpus Validation Results](#)
 - [20.1 Overview](#)
 - [20.2 Key Findings](#)
 - [20.3 Size Validation](#)
 - [20.4 PHYS Corpus Validation](#)
- [21. References](#)

1. Overview

M2 is the primary model format used by World of Warcraft for all 3D models rendered in-engine: player characters, creatures, spell visuals, doodads, weapons, and UI elements. It is used for everything except terrain (ADT) and world map objects (WMO).

An M2 model is not a single file. It is a **bundle** of related files, each carrying a different aspect of the model data:

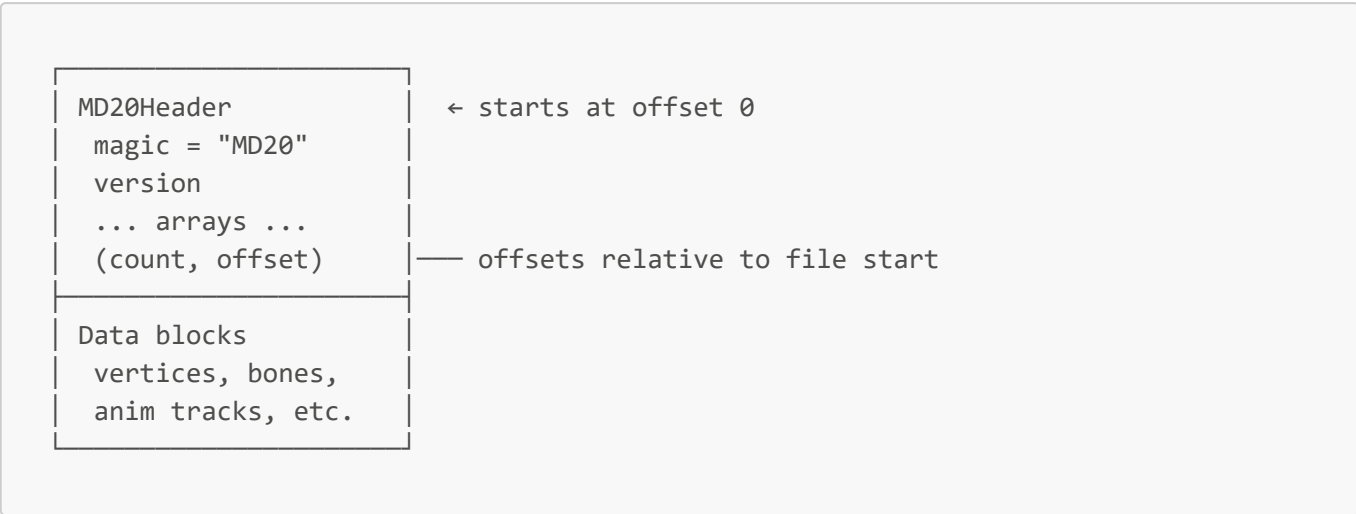
File	Purpose
.m2	Base model: header, geometry, bones, textures, materials, animations (or refs), cameras, lights, emitters, collision
.skin	LOD views: index buffers, submesh definitions, draw batches
.anim	Externalized animation keyframe data for low-priority sequences
.bone	Bone-specific data (face-pose offset matrices)
.skel	Chunked skeleton: bones, attachments, sequences delegated from a parent skeleton hierarchy
.phys	Ragdoll physics: rigid bodies, shapes, joints

WhiteoutLib supports parsing and writing all of the above except [.phys](#) (which is handled as inline raw data via the [PFDC](#) chunk).

2. Format Variants

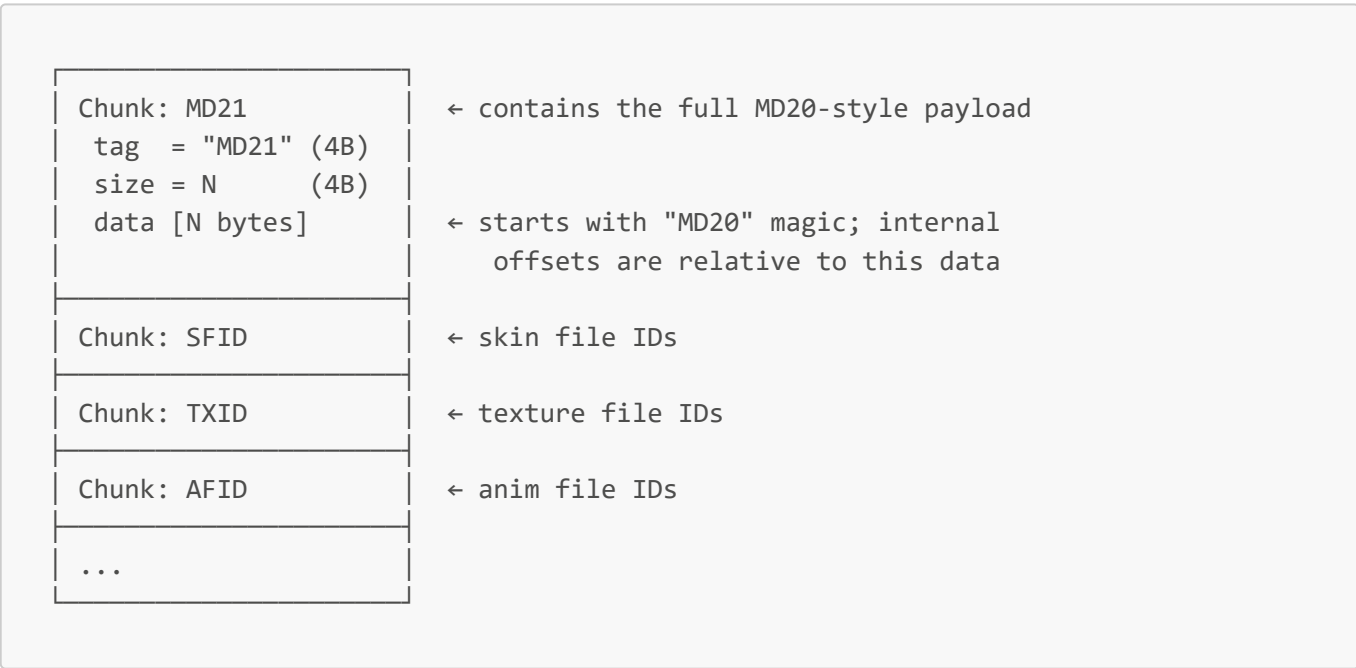
2.1 Classic (MD20)

Pre-Legion files begin directly with the four-byte magic MD20 (0x4D443230). The entire file is a flat binary blob: a fixed-layout header followed by variable-length data blocks addressed through offset/count pairs.



2.2 Chunked (MD21)

Starting with Legion (7.0.1), the .m2 file may use a chunked container. If the first four bytes are **not** MD20, the file is chunked. Chunks follow a uniform framing:



Important: Unlike every other chunked format in WoW, M2 chunk tags are **NOT byte-reversed** in the file. AFID in the file is 0x41464944, reading left-to-right as ASCII.

Chunks may appear in any order. The MD21 chunk is typically first.

3. Version History

The version field in the MD20 header encodes a major.minor pair as major * 256 + minor. WhiteoutLib defines these constants:

Constant	Value	Major.Minor	Expansion(s)
M2_VERSION_VANILLA	256	1.0	Classic / Pre-Release
M2_VERSION_BC	260	1.4	The Burning Crusade
M2_VERSION_WOTLK	264	1.8	Wrath of the Lich King
M2_VERSION_CATA	265	1.9	Cataclysm
M2_VERSION_MOP	272	1.16	Mists of Pandaria
M2_VERSION_WOD	272	1.16	Warlords of Draenor
M2_VERSION_LEGION	272	1.16	Legion
M2_VERSION_BFA	273	1.17	Battle for Azeroth
M2_VERSION_SL	274	1.18	Shadowlands

Version ranges are inclusive. Models are version-tagged by the exporter; the version determines which structural fields and parsing branches are active.

Corpus note: The 9059-file corpus contains only version 272 (3629 files) and version 274 (5430 files). All files use the chunked MD21 format — no classic MD20 files were present.

Version-dependent code paths in WhiteoutLib:

Gate	Effect
< WOTLK (264)	Skin profiles are inline in the <code>.m2</code> instead of external <code>.skin</code> files
>= WOTLK (264)	Header stores <code>numSkinProfiles</code> as <code>u32</code> instead of inline <code>M2Array<SkinProfile></code>
< CATA (265)	Camera has a flat <code>f32 fieldOfView</code> before far/near clip
>= CATA (265)	Camera has an <code>M2Track<f32> fieldOfViewTrack</code> after the roll track
<code>NewParticleRecord</code> flag OR > 271	<code>M2ParticleOld</code> struct is 492 bytes instead of 476
>= CATA	Particle emitter gains multi-texture extension fields (<code>M2Particle</code> wrapping <code>M2ParticleOld</code>)
<code>flag_use_texture_combiner_combos</code>	<code>textureCombinerCombos</code> array appended at header end

4. Primitive Types

WhiteoutLib defines these foundational types in `common_types.h`:

Scalar Types

Type	C++ Type	Size
------	----------	------

Type	C++ Type	Size
u8	uint8_t	1
u16	uint16_t	2
u32	uint32_t	4
u64	uint64_t	8
i8	int8_t	1
i16	int16_t	2
i32	int32_t	4
f16	uint16_t (stored)	2
f32	float	4
f64	double	8

Normalized Integers

Type	Range	Mapping
snorm8 / snorm16 / snorm32	[-1.0, 1.0]	Signed integer → float via division by max absolute value
unorm8 / unorm16 / unorm32	[0.0, 1.0]	Unsigned integer → float via division by max value

The M2 format makes extensive use of `fixed16` (same as `i16` / `snorm16`) for alpha and weight values: `0x0000` = 0.0, `0x7FFF` = 1.0.

Vector / Matrix Types

Type	Layout	Size
Vector2f	f32 x, y	8
Vector3f	f32 x, y, z	12
Vector4f	f32 x, y, z, w	16
Quaternion	f32 x, y, z, w	16
Matrix3x4	f32[12]	48
Matrix4x4	f32[16]	64

Compressed Quaternion (`CompatQuaternion`)

Post-Vanilla bone rotations use a compressed 8-byte quaternion:

```
struct CompatQuaternion {
    u16 x, y, z, w; // x,y,z are snorm16; w is unorm16
```

```
};
// Identity = (32767, 32767, 32767, 65535) → (0, 0, 0, 1)
```

XYZ components are signed normalized ($[-1, 1]$ mapped to $[0, 65534]$ with $32767 = 0$). The W component is unsigned normalized ($[0, 1]$ mapped to $[0, 65535]$).

Extent (Bounding Box + Sphere)

```
struct Extent {
    Vector3f minimum;      // 12 bytes
    Vector3f maximum;      // 12 bytes
    f32      sphereRadius; // 4 bytes
};                          // Total: 28 bytes
```

Used for both model bounding volumes and per-sequence animation bounds.

5. Core Patterns

5.1 M2Array

M2 data is organized through pointer-style arrays scattered throughout the file:

```
struct M2Array<T> {
    u32 count;    // number of elements
    u32 offset;   // byte offset to first element, relative to base
};               // Total: 8 bytes on disk
```

Offset base:

- **Classic MD20**: offsets are relative to the start of the file (offset 0)
- **Chunked MD21**: offsets are relative to the start of the MD21 chunk's **data** (after the 8-byte chunk header)

WhiteoutLib resolves these at parse time into `std::vector<T>`.

5.2 Animation Tracks

Animation data is the backbone of the M2 format. Nearly every animatable property uses this template:

```
struct M2TrackBase {
    u16 interpolationType; // 0=None, 1=Linear, 2=Bezier, 3=Hermite
    u16 globalSequenceId; // index into global_loops[], or 0xFFFF if none
    // >= Wrath: array-of-arrays
    M2Array<M2Array<u32>> timestamps; // [animation][keyframe] → ms
};
```

```
template<typename T>
struct M2Track : M2TrackBase {
    M2Array<M2Array<T>> values; // [animation][keyframe] → value
};
```

Each track has one sub-array per animation sequence. The outer index corresponds to the animation's position in the `sequences` array. If a sequence has no keyframes for this track, its sub-array is empty (count=0).

WhiteoutLib's in-memory representation:

```
struct AnimationTrackBase {
    u16 interpolationType;
    u16 globalSequenceId; // 0xFFFF = none
    std::vector<std::vector<u32>> timestamps;
};

template<typename T>
struct AnimationTrack : AnimationTrackBase {
    std::vector<std::vector<T>> values;
};
```

Pre-Wrath (< 264): Tracks use a single-timeline approach — one flat timestamp array and one flat value array, with an `interpolation_ranges` array mapping each animation to its begin/end indices. WhiteoutLib does not implement this path.

5.3 Interpolation

Type	Value	Behavior
None	0	Step function; value changes instantly at the timestamp
Linear	1	Linear interpolation (lerp for vectors/colors; nlerp for quaternions)
Bezier	2	Cubic Bézier spline using <code>M2SplineKey</code> (value + inTan + outTan)
Hermite	3	Cubic Hermite spline using <code>M2SplineKey</code>

For spline-based interpolation (types 2 and 3), keyframes use `SplineKey<T>`:

```
template<typename T>
struct SplineKey {
    u32 timestamp;
    T value;
    T inTangent;
    T outTangent;
};
```

Note: There is historical ambiguity about whether type 2 is Bézier or Hermite. From WotLK onwards, type 2 = Bézier is confirmed by the community.

5.4 Global Sequences

Global sequences provide animation timing that is completely independent of the model's current animation. They are used for perpetually looping effects (weapon glows, ambient light pulses, etc.).

```
struct M2Loop {
    u32 timestamp; // maximum time in milliseconds
};
```

A track with `globalSequenceId != 0xFFFF` ignores per-animation timelines and instead loops within `[0, global_loops[globalSequenceId].timestamp]`. Global-sequence tracks always have exactly one sub-array (index 0).

5.5 FBlock (Fake-AnimationBlock)

Particle emitters use a simplified animation block that cannot vary between animations:

Offset	Type	Field	
0x00	u32	nTimestamps	
0x04	u32	ofsTimestamps	→ fixed16 timestamps (u16!)
0x08	u32	nValues	
0x0C	u32	ofsValues	→ T values

These point directly to data without the outer per-animation indirection.

6. MD20 Header

6.1 Binary Layout

The header is a fixed-layout structure of count/offset pairs. Offsets point into the data block that follows the header. Record sizes are not stored — they are implicit from the structure definitions.

Offset	Type	Field
0x000	u32	magic ("MD20")
0x004	u32	version
0x008	M2Array<char>	name
0x010	u32	globalFlags
0x014	M2Array<M2Loop>	globalLoops
0x01C	M2Array<M2Sequence>	sequences
0x024	M2Array<u16>	sequenceIdxHashById
0x02C	M2Array<M2CompBone>	bones
0x034	M2Array<u16>	keyBoneLookup
0x03C	M2Array<M2Vertex>	vertices

0x044	u32	numSkinProfiles	(>= WotLK)
0x048	M2Array<M2Color>	colors	
0x050	M2Array<M2Texture>	textures	
0x058	M2Array<M2TextureWeight>	textureWeights	
0x060	M2Array<M2TextureTransform>	textureTransforms	
0x068	M2Array<u16>	textureIndicesById	
0x070	M2Array<M2Material>	materials	
0x078	M2Array<u16>	boneCombos	
0x080	M2Array<u16>	textureCombos	
0x088	M2Array<u16>	textureCoordCombos	
0x090	M2Array<u16>	textureWeightCombos	
0x098	M2Array<u16>	textureTransformCombos	
0x0A0	Extent	bounding	
0x0BC	Extent	collision	
0x0D8	M2Array<u16>	collisionTriangleIndices	
0x0E0	M2Array<Vector3f>	collisionVertices	
0x0E8	M2Array<Vector3f>	collisionFaceNormals	
0x0F0	M2Array<M2Attachment>	attachments	
0x0F8	M2Array<u16>	attachmentIndicesById	
0x100	M2Array<M2Event>	events	
0x108	M2Array<M2Light>	lights	
0x110	M2Array<M2Camera>	cameras	
0x118	M2Array<u16>	cameraIndicesById	
0x120	M2Array<M2Ribbon>	ribbonEmitters	
0x128	M2Array<M2Particle>	particleEmitters	
--- conditional (if globalFlags & UseTextureCombinerCombos, >= BC) ---			
0x130	M2Array<u16>	textureCombinerCombos	

WhiteoutLib stores this as **MD20Header**:

```
struct MD20Header {
    u32 magic;           // MD20
    u32 version;
    std::string modelName;
    GlobalFlags globalFlags;

    std::vector<GlobalSequence> globalLoops;
    std::vector<Sequence> sequences;
    std::vector<u16> sequenceIdxHashById;

    std::vector<Bone> bones;
    std::vector<u16> keyBoneIds;
    std::vector<Vertex> vertices;
    u32 numSkinProfiles;

    std::vector<ColorAnimation> colors;
    std::vector<Texture> textures;
    std::vector<TextureWeight> textureWeights;
    std::vector<TextureTransform> textureTransforms;
    std::vector<u16> textureIndicesById;
    std::vector<Material> materials;
```

```
std::vector<u16> boneCombos;
std::vector<u16> textureCombos;
std::vector<u16> textureCoordCombos;
std::vector<u16> textureWeightCombos;
std::vector<u16> textureTransformCombos;

Extent bounding;
Extent collision;
std::vector<u16> collisionTriangleIndices;
std::vector<Vector3f> collisionVertices;
std::vector<Vector3f> collisionFaceNormals;

std::vector<Attachment> attachments;
std::vector<u16> attachmentIndicesById;
std::vector<Event> events;
std::vector<Light> lights;
std::vector<Camera> cameras;
std::vector<u16> cameraIndicesById;
std::vector<RibbonEmitter> ribbonEmitters;
std::vector<ParticleEmitter> particleEmitters;
std::vector<u16> textureCombinerCombos; // if flag set
};
```

6.2 Global Flags

Stored as `u32` at offset `0x010` in the header. WhiteoutLib exposes these as `GlobalFlag`:

Flag	Bit	Description	Corpus Freq
TiltX	0x001	Model tilts along X axis	402 / 9059
TiltY	0x002	Model tilts along Y axis	118 / 9059
Unk_0x04	0x004	Unknown	60 / 9059
UseTextureCombinerCombos	0x008	Appends <code>textureCombinerCombos</code> array at header end; multitexturing uses second material from this array	—
Unk_0x10	0x010	Unknown; extremely common in modern exports, purpose unclear	5664 / 9059
LoadPhysicsData	0x020	Model has associated physics data (<code>.phys</code> or <code>PFDC</code> chunk)	56 / 9059

Flag	Bit	Description	Corpus Freq
Unk_0x80	0x080	Universal in modern (10.x/11.x/12.x) files; originally described as "DH tattoo glow" but set on every model in the corpus regardless of model type	9059 / 9059
CameraRelated	0x100	Camera-related flag; referenced by wowdev.wiki as "every model since Cata has this set", but corpus shows only 3 of 9059 files have it — likely only present on models with actual camera definitions	3 / 9059
NewParticleRecord	0x200	Extended 492-byte particle record (instead of 476 bytes)	7 / 9059
Unk_0x400	0x400	Unknown	—
TextureTransformsUsesBoneSequences	0x800	Texture transforms use the sequence played on a bone (looked up via <code>textureCoordCombos</code>) instead of the model's first bone	—
Unk_0x1000	0x1000	Unknown	2617 / 9059
ChunkedAnimFiles	0x2000	<code>.anim</code> files use chunked <code>AFM2/AFSA/AFSB</code> format	5867 / 9059
Unk_0x8000	0x8000	Unknown	512 / 9059
Unk_0x20000	0x20000	Unknown; may relate to ribbon emitter <code>textureTransformLookupIndex</code> usage (see ribbon emitter field documentation)	560 / 9059
Unk_0x100000	0x100000	Unknown	61 / 9059
Unk_0x200000	0x200000	Unknown; very common in modern files	4274 / 9059
Unk_0x1000000	0x1000000	Unknown	21 / 9059

Flag	Bit	Description	Corpus Freq
Unk_0x20000000	0x20000000	Unknown	53 / 9059
Unk_0x40000000	0x40000000	Unknown	14 / 9059

Bits not listed above (0x4000, 0x10000, 0x40000, 0x80000, etc.) were not observed in the 9059-file corpus but are preserved as raw flags by WhiteoutLib.

7. Skeleton and Animation

7.1 Global Sequence Loops

```
struct GlobalSequence {
    u32 timestamp; // maximum loop time in milliseconds
};
```

Referenced by tracks via `globalSequenceId`. If a track references a global sequence, its animation is independent of which model animation is playing.

7.2 Animation Sequences

```
struct Sequence {
    u16 id; // animation ID (references AnimationData.dbc)
    u16 variationIndex; // sub-animation index within the same ID
    u32 duration; // length in milliseconds (>= Wrath)
    f32 movespeed; // character movement speed during this animation
    u32 flags; // see Sequence Flags
    i16 frequency; // playback probability weight; all variations of an
    // ID sum to 0x7FFF (32767)

    u16 padding;
    u32 replayMin; // minimum repetition count (0 = no repeat)
    u32 replayMax; // maximum repetition count
    u16 blendTimeIn; // transition blend-in duration (ms)
    u16 blendTimeOut; // transition blend-out duration (ms)
    Extent bounding; // per-sequence bounding box
    i16 variationNext; // index of next variation of this animation ID, or -1
    u16 aliasNext; // if this is an alias (flag 0x40), index of target
}; // Total: 68 bytes
```

The `blendTimeIn` / `blendTimeOut` split was introduced around WoD; older files had a single `u32 blendTime` in this slot. The client interpolates bone transforms between the ending state of one animation and the starting state of the next for the duration of the blend time.

7.3 Sequence Flags

Flag	Value	Description	Corpus Freq
Unk_0x01	0x01	Unknown; seen commonly in sequences	19220 / 45310
Tilt In	0x02	Model starts upright, tilts over X/Y by end	21305 / 45310
Tilt Out	0x04	Model starts tilted, returns upright by end	21062 / 45310
Tilt Fixed	0x08	Model stays tilted for entire animation	20664 / 45310
Runtime loaded	0x10	Set at runtime for loaded low-priority sequences	21310 / 45310
Primary bone sequence	0x20	Animation data is embedded in the .m2 file (not externalized to .anim)	21583 / 45310
Is alias	0x40	This entry is an alias; follow aliasNext to find actual data	20528 / 45310
Blended	0x80	Lerp between end→start states on transition if values differ	22209 / 45310
Stored in model	0x100	Sequence data is stored within the model	19852 / 45310
Dual blend time	0x200	The 4-byte blend time field is two u16 (in + out)	19823 / 45310
Unk_0x400	0x400	Unknown	18908 / 45310
Unk_0x800	0x800	Seen in Legion 24500 models	19622 / 45310

Special flag patterns: The value **0x7FFF** (all bits 0–14 set) appears in 6347 sequences; these are typically alias sequences with all standard flags enabled. The value **0xFFFFFFFF** (-1 as **i32**) appears in 473 sequences and may serve as a sentinel. When the upper 16 bits of the flags **u32** are non-zero, the lower 16 bits are almost always **0xFFFF**, suggesting the upper half may carry auxiliary data (possibly a parent animation ID) rather than additional flags.

Corpus note: Across 9059 files (45310 total sequences), 10919 unique flag values were observed. 69% of sequences (31280) have the upper 16 bits as zero.

External animation loading rule: The client loads **.anim** files when **(flags & 0x130) == 0** — i.e., the animation is neither primary-bone-embedded (0x20), nor stored in model (0x100), nor has 0x10 set.

7.4 Sequence Index Hash (Animation Lookup)

```
u16 sequenceIdxHashById[]; // length = hash table size
```

A hash table mapping animation IDs to indices in the `sequences` array. The hash function is `anim_id % num_buckets` with quadratic probing (`stride2` increments). Value `-1` (0xFFFF) means "no entry". The referenced sequence is the first in its `aliasNext` chain for the given animation ID.

```
M2Sequence* find_entry(u32 anim_id) {
    size_t i = anim_id % sequenceIdxHashById.size();
    for (size_t stride = 1; ; ++stride) {
        if (sequenceIdxHashById[i] == 0xFFFF)
            return nullptr;
        if (sequences[sequenceIdxHashById[i]].id == anim_id)
            return &sequences[sequenceIdxHashById[i]];
        i = (i + stride * stride) % sequenceIdxHashById.size();
    }
}
```

7.5 Bones

```
struct Bone {
    i32 keyBoneId;           // index in key_bone_lookup, or -1 if not a key bone
    u32 flags;               // BoneFlag bits
    i16 parentBoneId;        // parent index, or -1 for root
    u16 submeshId;           // mesh part ID
    u32 boneNameCRC;         // CRC of bone name (debug only)

    AnimationTrack<Vector3f> translation;
    AnimationTrack<CompatQuaternion> rotation; // compressed; Vanilla used
                                                    // full C4Quaternion

    AnimationTrack<Vector3f> scale;
    Vector3f pivot;          // pivot point in model space
};
```

The bone hierarchy defines the skeleton. Each bone's final transform is computed as `parent_transform * translate(pivot) * T * R * S * translate(-pivot)`, where T/R/S come from the animation tracks.

Coordinate system: WoW uses a Z-up coordinate system. To convert to Y-up: $(x, y, z) \rightarrow (x, -z, y)$.

7.6 Bone Flags

Flag	Value	Description
IgnoreParentTranslate	0x001	Don't inherit parent's translation

Flag	Value	Description
IgnoreParentScale	0x002	Don't inherit parent's scale
IgnoreParentRotation	0x004	Don't inherit parent's rotation
SphericalBillboard	0x008	Bone always faces the camera (all axes)
CylindricalBillboardX	0x010	Billboard locked to X axis
CylindricalBillboardY	0x020	Billboard locked to Y axis
CylindricalBillboardZ	0x040	Billboard locked to Z axis
Transformed	0x200	Bone has animation transforms
Kinematic	0x400	Physics system can influence this bone (MoP+)
HelmetAnimScaled	0x1000	Helmet animation scaling via HelmetAnimScaling.dbc

Spherical and cylindrical billboard bits are mutually exclusive. Billboards are used for light halos, cannonball hemispheres, and similar view-facing geometry.

7.7 Key-Bone Lookup

```
u16 keyBoneLookup[]; // -1 (0xFFFF) if not present
```

Maps well-known bone roles to bone indices. The lookup index corresponds to a predefined bone role:

Index	Bone Role	Index	Bone Role
0	ArmL	1	ArmR
2	ShoulderL	3	ShoulderR
4	SpineLow	5	Waist
6	Head	7	Jaw
8–12	IndexFingerR through ThumbR	13–17	IndexFingerL through ThumbL
26	Root	27–34	Wheel1–8

Most character/creature models have 27–35 entries.

8. Geometry and Rendering

8.1 Vertices

```
struct Vertex {
    Vector3f    position;           // 12 bytes
    u8          boneWeights[4];    // 4 bytes (sum should be 255)
```

```
    u8          boneIndices[4]; // 4 bytes (into bones[])
    Vector3f    normal;         // 12 bytes
    Vector2f    texCoords[2];   // 16 bytes (two UV sets)
};                               // Total: 48 bytes
```

Vertices are stored globally in the `.m2` file. Skin profiles reference subsets of this vertex array. The two UV sets support multi-texturing; which set is used depends on the vertex shader selected by the rendering pipeline (e.g., `Diffuse_T1_T2` sends both sets, `Diffuse_T1_T1` sends the same UV for both textures).

8.2 Textures

```
struct Texture {
    u32          type;          // see Texture Types
    u32          flags;         // 0x1 = wrap X, 0x2 = wrap Y
    M2Array<char> filename;     // for type 0 only; includes null terminator
};
```

Type 0 textures have an in-file filename (BLP path). All other types are "hardcoded" — their actual texture is determined at runtime from database lookups (`CharSections.dbc`, `CreatureDisplayInfo.dbc`, `ItemDisplayInfo.dbc`, etc.). Non-type-0 textures still have a 1-byte filename pointing to a null terminator.

In chunked files ($\geq$ BfA), the `TXID` chunk provides `fileDataIDs` for textures, replacing filenames. The filename `M2Array` should be zeroed so the client falls through to the `TXID` lookup.

Implementation note: The filename buffer is stack-allocated at 0x108 bytes in the client, imposing an implicit 264-character path length limit.

8.3 Texture Types

Value	Name	Usage
0	NONE	Explicit filename / fileDataID
1	TEX_COMPONENT_SKIN	Body + clothes
2	TEX_COMPONENT_OBJECT_SKIN	Item, cape
3	TEX_COMPONENT_WEAPON_BLADE	Weapon blade / armor reflect
4	TEX_COMPONENT_WEAPON_HANDLE	Weapon handle
5	TEX_COMPONENT_ENVIRONMENT	Obsolete
6	TEX_COMPONENT_CHAR_HAIR	Character hair
7	TEX_COMPONENT_CHAR_FACIAL_HAIR	Obsolete
8	TEX_COMPONENT_SKIN_EXTRA	Skin extra detail
9	TEX_COMPONENT_UI_SKIN	Inventory art
10	TEX_COMPONENT_TAUREN_MANE	Character misc

Value	Name	Usage
11–13	TEX_COMPONENT_MONSTER_1–3	Creature/GO skins
14	TEX_COMPONENT_ITEM_ICON	Item icon
15–18	Guild background/emblem/border colors	> = Cata
19	Character Eyes	> = SL
20	Character Jewelry / Accessory	> = SL
21–24	Secondary skin/hair/armor + unknown	> = SL (not observed in 10.x/11.x corpus)
25–26	Unknown	> = DF (type 25: 4 files, type 26: 2 files in corpus)

8.4 Materials (Render Flags)

```
struct Material {
    u16 flags;           // see below
    u16 blendingMode;    // see M2/Rendering#M2BLEND
};
```

Material Flags:

Flag	Value	Meaning
Unlit	0x01	Not affected by lighting
Unfogged	0x02	Not affected by fog
TwoSided	0x04	Disable backface culling
DepthTest	0x08	Enable depth testing
DepthWrite	0x10	Enable depth writing
ShadowBatch1	0x40	Shadow batch related (WoD+)
ShadowBatch2	0x80	Shadow batch related (WoD+)
ShadowBatch3	0x200	Shadow batch related (WoD+)
NoAlphaComposite	0x800	Force opaque or fully transparent for custom elements (MoP+)

Blending modes (from M2/Rendering#M2BLEND):

Value	OpenGL Equivalent	Corpus Freq
0	glDisable(GL_BLEND); glDisable(GL_ALPHA_TEST)	2383
1	glBlendFunc(GL_SRC_COLOR, GL_ONE)	824
2	glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)	7354

Value	OpenGL Equivalent	Corpus Freq
3	<code>glDisable(GL_BLEND); glEnable(GL_ALPHA_TEST)</code>	—
4	<code>glBlendFunc(GL_SRC_ALPHA, GL_ONE)</code>	8743
5	Unknown (possibly <code>GL_DST_COLOR * ONE_MINUS_SRC_ALPHA</code>)	6
6	Unknown (possibly multiplicative or screen blend)	42
7	Unknown; very common in modern files — likely a Shadowlands+ blend mode	4133

8.5 Colors and Transparency

Color animations provide per-vertex color and alpha modulation:

```
struct ColorAnimation {
    AnimationTrack<Vector3f> color;    // RGB, [0,1] per component
    AnimationTrack<i16>      alpha;    // 0 = transparent, 0x7FFF = opaque
};
```

Texture weights specify global transparency in addition to color alpha:

```
struct TextureWeight {
    AnimationTrack<i16> weight; // 0 = transparent, 0x7FFF = opaque
};
```

Both are referenced indirectly from skin batches via the combo/lookup tables. If a batch's `colorIndex` is -1, the submesh does not use this block.

8.6 Texture Transforms

```
struct TextureTransform {
    AnimationTrack<Vector3f>      translation;
    AnimationTrack<CompatQuaternion> rotation;    // center = (0.5, 0.5)
    AnimationTrack<Vector3f>      scaling;
};
```

Used for animated textures: flowing water, lava, pulsing runes, etc. The rotation is applied around the texture center `(0.5, 0.5)`:

1. Translate UV matrix to `(0.5, 0.5)`
2. Apply rotation
3. Translate UV matrix to `(-0.5, -0.5)`

8.7 Lookup / Combo Tables

The header contains several `u16[]` indirection tables that skin batches use to reference model-level resources:

Table	Alt Name	Purpose
<code>boneCombos</code>	<code>bone_lookup_table</code>	Bone indices for skinning; skin sections reference a slice
<code>textureCombos</code>	<code>texture_lookup_table</code>	Indices into the <code>textures[]</code> array
<code>textureCoordCombos</code>	<code>tex_unit_lookup_table</code>	UV mapping selection (-1=env, 0=UV0, 1=UV1)
<code>textureWeightCombos</code>	<code>transparency_lookup_table</code>	Indices into <code>textureWeights[]</code>
<code>textureTransformCombos</code>	<code>texture_transforms_lookup_table</code>	Indices into <code>textureTransforms[]</code>
<code>textureIndicesById</code>	<code>replacable_texture_lookup</code>	Replaceable texture lookup (type → texture index, or -1)
<code>textureCombinerCombos</code>	<code>Second_Texture_Material_Override_Combos</code>	Second-pass material override for multitexturing (flag-gated)

When `UseTextureCombinerCombos` is set, multitexturing reads the second texture's blend mode from `textureCombinerCombos[batch.shaderId]` instead of from `materials[batch.materialIndex + 1]`.

For models with multitexturing, lookup tables function in pairs: the first texture uses the mapping at the given index, the second texture uses the next index in sequence.

9. Bounding and Collision Volumes

The header `bounding` and `collision Extent` structures define the overall model bounds and a simplified collision volume.

Collision geometry is defined by three arrays:

Array	Content
<code>collisionVertices[]</code>	<code>Vector3f</code> positions
<code>collisionTriangleIndices[]</code>	<code>u16</code> indices (3 per triangle, count = number of indices)
<code>collisionFaceNormals[]</code>	<code>Vector3f</code> normals (1 per triangle, count = indices/3)

Characters and creatures typically have a simple box-like collision mesh.

10. Scene Objects

10.1 Attachments

```
struct Attachment {
    u32 id; // attachment type (see table below)
    u16 boneId; // anchor bone
    u16 unknown;
    Vector3f position; // offset from bone; often same as bone pivot
    AnimationTrack<u8> animate; // enable/disable flag (bool); default = true
};
```

Attachments define mount points for weapons, effects, and UI elements on the model. The `id` determines the semantic role:

Attachment ID table (selected):

ID	Name	ID	Name	ID	Name
0	Shield / MountMain	11	Helm	26	SheathMainHand
1	HandRight	12	Back	27	SheathOffHand
2	HandLeft	17	Breath	28	SheathShield
5	ShoulderRight	18	PlayerName	29	PlayerNameMounted
6	ShoulderLeft	19	Base	34	Chest
7–10	KneeR/L, HipR/L	20	Head	39–46	VehicleSeat1–8 (Wrath+)
3	ElbowRight	21	SpellLeftHand	47	LeftFoot
4	ElbowLeft	22	SpellRightHand	48	RightFoot

For weapons, IDs 0–4 correspond to the 5 `ItemVisuals.dbc` columns for glow effects. The `PlayerName` (18) attachment is used by `CGUnit_C::GetNamePosition` for the name plate. `PlayerNameMounted` (29) takes priority when mounted.

The `attachmentIndicesById` lookup maps attachment type → index in `attachments[]`. Value -1 means not present.

10.2 Events

```
struct Event {
    u32 identifier; // 4-char ID, usually '$' + 3 chars (e.g., "$DTH")
    u32 data; // context-dependent payload (often soundEntryId)
    u32 boneId; // attachment bone
    Vector3f position; // relative to bone
    AnimationTrackBase enabled; // timestamp-only track; each timestamp = "fire"
};
```

Events carry no value data — each timestamp in the `enabled` track is an implicit trigger. The `identifier` is a 4-byte ASCII code (stored as `u32`), and `data` carries a context-dependent payload (e.g., `soundEntryId` for

sound events, `SpellEffectCameraShakes.dbc` ID for screen shakes).

Common events:

ID	Purpose	Triggered On
<code>\$DTH</code>	Death thud + loot effect	Death, Drown, Knockdown
<code>\$FSD</code>	Footstep sound	Walk, Run
<code>\$CST</code>	Release missiles	Attack/Spell casts
<code>\$CSL / \$CSR</code>	Release missiles left/right	Attack/Spell casts
<code>\$BWR</code>	Bow release	AttackRifle, AttackBow
<code>\$BWP</code>	Bow pull	LoadRifle, LoadBow
<code>\$SHL / \$SHR</code>	Sheathe weapon left/right	Sheath, HipSheath
<code>\$BTH</code>	Breath effect	Idle (underwater, snow)
<code>\$CSS</code>	Weapon swoosh sound	Attack animations
<code>\$HIT</code>	Wound anim kit	Attack animations
<code>\$FL0-\$FL3</code>	Footstep left	Walk/Run
<code>\$FR0-\$FR3</code>	Footstep right	Walk/Run
<code>\$CSD</code>	Emote sound (data=soundEntryId)	Emote animations
<code>\$DSE</code>	Destroy emitter	Map objects
<code>\$DSO</code>	Doodad sound one-shot (data=soundEntryId)	Game objects
<code>\$SHK</code>	Camera shake (data=cameraShakeId)	Game objects

10.3 Lights

```
struct Light {
    u16 type;           // 0 = Directional, 1 = Point
    i16 boneId;         // -1 if not bone-attached
    Vector3f position;  // relative to bone

    AnimationTrack<Vector3f> ambientColor;
    AnimationTrack<f32>      ambientIntensity; // default 1.0
    AnimationTrack<Vector3f> diffuseColor;
    AnimationTrack<f32>      diffuseIntensity; // default 1.0
    AnimationTrack<f32>      attenuationStart;
    AnimationTrack<f32>      attenuationEnd;
    AnimationTrack<u8>        visibility;      // 0/1 toggle
};
```

Type	Value	Usage
Directional	0	Login screens only (not used in world)
Point	1	Wands, doodads, creature effects

In TBC (and possibly other versions), the client hardcodes constant attenuation to 0, linear attenuation to 0.7, and quadratic attenuation to 0.03 for login screen lights.

10.4 Cameras

```
struct Camera {
    u32 type;           // 0=portrait, 1=characterinfo, -1=flyby
    f32 fieldOfView;    // diagonal FOV in radians (< Cata only)
    f32 farClip;
    f32 nearClip;

    AnimationTrack<CameraSpline> positions;
    Vector3f positionBase;
    AnimationTrack<CameraSpline> targetPositions;
    Vector3f targetPositionBase;
    AnimationTrack<f32> roll;           // 0 to 2π
    AnimationTrack<f32> fieldOfViewTrack; // >= Cata only
};

struct CameraSpline {
    Vector3f value;
    Vector3f inTangent;
    Vector3f outTangent;
};
```

Cameras are present in most models for the character portrait and character info tab. Flyby cameras (type -1) are used in geometry-less camera-path models and login screen backdrops.

FOV conversion: The M2 stores diagonal FOV (d_{fov}). Convert to vertical FOV (v_{fov}):

$$v_{fov} = \frac{d_{fov}}{\sqrt{1 + aspect^2}}$$

The `cameraIndicesById` lookup maps type → camera index. `cameraIndicesById[1]` is the character info tab camera. A lookup value of -1 means "not referenced."

11. Effects

11.1 Ribbon Emitters

```
struct RibbonEmitter {
    u32 ribbonId;           // usually -1
    u32 boneId;
    Vector3f position;      // relative to bone
};
```

```

std::vector<u16> textureIndices; // into textures[]
std::vector<u16> materialIndices; // into materials[]

AnimationTrack<Vector3f> colorTrack; // RGB multiplier
AnimationTrack<i16> alphaTrack; // 0=transparent, 0x7FFF=opaque
AnimationTrack<f32> heightAboveTrack;
AnimationTrack<f32> heightBelowTrack;

f32 edgesPerSecond; // smoothness; quads generated per second
f32 edgeLifetime; // seconds quads persist
f32 gravity; // arcsin(val) = emission angle in degrees

u16 textureRows; // texture atlas rows
u16 textureCols; // texture atlas columns

AnimationTrack<u16> texSlotTrack;
AnimationTrack<u8> visibilityTrack;

i16 priorityPlane; // >= Wrath
i8 ribbonColorIndex;
i8 textureTransformLookupIndex; // into textureTransformCombos;
// only used if globalFlags & 0x20000
};

```

Ribbons generate trailing quads along the path of the attached bone. **heightAbove/heightBelow** define the ribbon width (they should differ). **edgesPerSecond** controls smoothness (low value = more jagged), and **edgeLifetime** controls trail length.

Examples: wisps in Blackfathom Deeps, Al'ar the Phoenix, Caverns of Time entrance light-trails.

11.2 Particle Emitters

Particle emitters are the most complex M2 structure. The base layout (**M2ParticleOld**, 476 or 492 bytes) contains:

```

struct ParticleEmitter {
    u32 particleId; // usually -1
    u32 flags; // see Particle Flags
    Vector3f position; // relative to bone
    u16 boneId;
    u16 texture; // or Cata+ bitfield: 5+5+5+1 bits for 3
textures

    std::string geometryModelFilename; // spawned model geometry
    std::string recursionModelFilename; // alias for up to 4 emitters of target

    u8 blendingType; // 0-4 (see Particle Blendings)
    u8 emitterType; // 1=Plane, 2=Sphere, 3=Spline, 4=Bone
    u16 particleColorIndex; // 11/12/13 → ParticleColor.dbc row

```

```

    u16 textureTileRotation;          // -1, 0, or 1 – also serves as priorityPlane
    u16 textureDimensions_rows;
    u16 textureDimensions_columns;

    // Animated tracks:
    AnimationTrack<f32> emissionSpeed;
    AnimationTrack<f32> speedVariation;          // [0, 1]
    AnimationTrack<f32> verticalRange;           // [0,  $\pi$ ]
    AnimationTrack<f32> horizontalRange;         // [0,  $2\pi$ ]
    AnimationTrack<f32> gravity;
    AnimationTrack<f32> lifespan;
    f32 lifespanVary;                          //  $\geq$  Wrath; particle.life += lifespanVary *
random(-1,1)
    AnimationTrack<f32> emissionRate;
    f32 emissionRateVary;                      //  $\geq$  Wrath
    AnimationTrack<f32> emissionAreaLength;      // plane: X width, sphere: min
radius
    AnimationTrack<f32> emissionAreaWidth;       // plane: Y width, sphere: max
radius
    AnimationTrack<f32> zSource;

    // FBlock color/alpha/scale ( $\geq$  Wrath):
    FBlock<Vector3f> colorTrack; // start, middle, end
    FBlock<i16>      alphaTrack;
    FBlock<Vector2f> scaleTrack;
    Vector2f        scaleVary;

    FBlock<u16>      headCellTrack;
    FBlock<u16>      tailCellTrack;

    f32 tailLength;
    f32 twinkleSpeed;          // twinkleFPS
    f32 twinklePercent;
    CRange twinkleScale;       // {min, max}
    f32 burstMultiplier;       // requires flag 0x40
    f32 drag;                   // speed *= exp(-drag * t)

    f32 baseSpin, baseSpinVary; //  $\geq$  Wrath
    f32 spin, spinVary;          // rotation per second

    M2Box tumble;               // {min, max} rotation speed vectors
    Vector3f windVector;
    f32 windTime;

    f32 followSpeed1, followScale1;
    f32 followSpeed2, followScale2;

    std::vector<Vector3f> splinePoints; // for spline emitter type
    AnimationTrack<u8> enabledIn;        // visibility toggle
};

```

Particle types:

Value	Description
0	Normal billboarded particle
1	Large quad from origin to position (Moonwell water)
2	Appears same as 0

Blending modes:

Value	Blend
0	Opaque (no blend, no alpha test)
1	<code>SRC_COLOR + ONE</code>
2	<code>SRC_ALPHA + ONE_MINUS_SRC_ALPHA</code> (standard alpha)
3	Alpha test (no blend)
4	<code>SRC_ALPHA + ONE</code> (additive)

Compressed Particle Gravity (flag `0x800000`): Gravity keyframe values are stored as 4 bytes (`{i8 x, i8 y, i16 z}`) instead of a float. At load time:

```
CompressedParticleGravity v = *(CompressedParticleGravity*)pValue;
Vector3f dir = Vector3f(v.x, v.y, 0) * (1.0f / 128.0f);
float z = sqrtf(1.0f - dot(dir, dir));
float mag = v.z * 0.04238648f;
if (mag < 0) { z = -z; mag = -mag; }
dir.z = z;
dir *= mag;
```

Without the flag, gravity is a simple `float` applied as `(0, 0, -gravity)`.

11.3 Particle Flags

Flag	Value	Description
Affected by lighting	<code>0x1</code>	Particle lit by scene lights
Player orientation	<code>0x4</code>	Initial orientation affected by player facing
World-space up	<code>0x8</code>	Particles travel up in world space, not model space
Don't trail	<code>0x10</code>	Disable particle trails
Unlit	<code>0x20</code>	Not affected by lighting
Use burst multiplier	<code>0x40</code>	Apply <code>burstMultiplier</code> to initial velocity
Model space	<code>0x80</code>	Particles stay in model space (emitter animation carried to particles)

Flag	Value	Description
Pinned	0x400	Quad expands from creation point
XY Quad	0x1000	Align to XY plane facing Z
Clamp to ground	0x2000	Project particles to ground plane
Choose random texture	0x10000	Random texture tile selection
Outward	0x20000	Particles move away from origin
Inward	0x40000	Particles move toward origin (usually opposite of 0x20000)
Scale vary independent	0x80000	X and Y scale vary independently
Random flipbook start	0x200000	Start texture animation at random frame
Ignore distance	0x400000	Don't throttle based on camera distance
Compressed gravity	0x800000	Gravity values use compressed 4-byte format
Bone generator	0x1000000	Bone, not joint
No distance throttle	0x4000000	Do not throttle emission rate based on distance
Multi-texture	0x10000000	Multi-textured particle (Cata+)

12. SKIN File (.skin)

Skin files define Level-of-Detail (LOD) views onto the model's vertex data. Starting from WotLK, they are stored externally; earlier versions embedded them in the .m2 file. Each .skin file contains a single SkinProfile.

12.1 Skin Profile

The header is 64 bytes for Cata+ models (48 bytes pre-Cata):

Offset	Size	Type	Field
-----	----	----	-----
0	4	u32	magic ("SKIN" = 0x534B494E)
4	8	M2Array<u16>	vertices
12	8	M2Array<u16>	indices
20	8	M2Array<u8[4]>	bones
28	8	M2Array<SkinSection>	submeshes
36	8	M2Array<Batch>	batches
44	4	u32	lodVertexBase
48	8	M2Array<ShadowBatch>	shadowBatches (>= Cata)
56	4	u32	reserved (always 0) (>= Cata)
60	4	u32	reserved (always 0) (>= Cata)

```
struct SkinProfile {
    u32 magic; // "SKIN" (0x534B494E)

    std::vector<u16> vertices; // indices into M2 vertex array
    std::vector<u16> indices; // triangle indices into vertices[]
    std::vector<std::array<u8,4>> bones; // per-vertex bone remapping
    std::vector<SkinSection> submeshes;
    std::vector<Batch> batches;

    u32 lodVertexBase; // cumulative vertex base for LOD skins (always 0 for skin00)

    std::vector<ShadowBatch> shadowBatches; // >= Cata
    // 8 bytes reserved (always 0)
};
```

Offsets within a `.skin` file are relative to the start of the file. The `vertices` array is a level of indirection: `skinProfile.indices[i]` is an index into `skinProfile.vertices[]`, and `skinProfile.vertices[j]` is an index into the M2's global `vertices[]`.

lodVertexBase — HD Model Vertex Splitting

All skin files use **u16** vertex indices and **u16** triangle indices — there is no u32 variant. Models with more than 65,535 total M2 vertices (e.g., HD character models with 200k–360k vertices) split their vertex data across multiple LOD skins, with each skin referencing a window of ≤65,535 vertices. The `lodVertexBase` field provides the offset into the M2's global vertex array for each skin:

- **Base skin (skin00):** `lodVertexBase = 0`. Vertex indices reference M2 global vertices `[0, vertexCount)`.
- **LOD skin N:** `lodVertexBase` = sum of `vertexCount` from skins 0 through N–1. Vertex indices reference M2 global vertices `[lodVertexBase, lodVertexBase + vertexCount)`.

Example — dracthyrfemale (359,270 total M2 vertices):

Skin File	lodVertexBase	Skin Verts	Global Range
skin00	0	61,934	0 – 61,933
lod01	61,934	61,934	61,934 – 123,867
lod02	123,868	55,407	123,868 – 179,274
lod03	179,275	50,391	179,275 – 229,665
lod04	229,666	46,263	229,666 – 275,928
lod05	275,929	42,342	275,929 – 318,270
lod06	318,271	40,999	318,271 – 359,269

Corpus (14,443 skin files): 6,779 non-LOD skins all have `lodVertexBase = 0`; all 4,867 LOD skins have `lodVertexBase > 0` matching the cumulative vertex chain.

12.2 Skin Sections (Submeshes)

```
struct SkinSection {
    u16 skinSectionId;    // submesh group identifier
    u16 level;            // LOD level (0 = highest detail)
    u16 vertexStart;      // start in skin's vertices[]
    u16 vertexCount;
    u16 indexStart;       // start in skin's indices[]
    u16 indexCount;
    u16 boneCount;        // bones used by this submesh
    u16 boneComboIndex;    // start in header's boneCombos[]
    u16 boneInfluences;    // max bones per vertex (1-4)
    u16 centerBoneIndex;   // bone for distance calculation
    Vector3f centerPosition;
    Vector3f sortCenterPosition;
    f32 sortRadius;
};
```

The `skinSectionId` groups submeshes for geoset-based visibility toggling (e.g., hairstyles use different geoset IDs, and only one is active at a time based on equipped items or character options).

12.3 Batches (Texture Units)

```
struct Batch {           // 24 bytes
    u8  flags;
    i8  priorityPlane;
    u16 shaderId;
    u16 skinSectionIndex;    // into submeshes[]
    u16 geosetIndex;
    i16 colorIndex;          // into header's colors[], or -1
    u16 materialIndex;       // into header's materials[]
    u16 materialLayer;
    u16 textureCount;
    u16 textureComboIndex;    // into header's textureCombos[]
    u16 textureCoordComboIndex; // into header's textureCoordCombos[]
    u16 textureWeightComboIndex;
    u16 textureTransformComboIndex;
};
```

Each batch represents a draw call. The `shaderId` selects which vertex/pixel shader combination to use and which UV sets to pass. When `UseTextureCombinerCombos` is set, `shaderId` also indexes into the `textureCombinerCombos` array for the second texture's blend mode.

Material, texture, and animation indices use the combo tables as indirection layers, enabling multiple skin sections to share resources efficiently.

Batch entries are always 24 bytes. The batch data array is followed by standard 16-byte alignment padding before the shadow batch data. For odd batch counts, $\text{batch_count} * 24 \equiv 8 \pmod{16}$, producing 8 bytes

of zero padding. This padding should not be confused with an extended 32-byte batch element size — there are no 32-byte batch entries in the corpus.

12.4 Shadow Batches

```
struct ShadowBatch {
    u8  flags;
    u8  flags2;          // 0x8 = use EDGF edge fade data
    u16 unknown0;
    u16 submeshId;
    u16 textureId;
    u16 colorId;
    u16 transparencyId;
};
```

Present from Cata onwards. Defines the shadow rendering pass for submeshes. `flags2 & 0x8` triggers edge-fade rendering using the `EDGF` chunk. Shadow batch data immediately follows the batch data region in the file.

Corpus (3,968 files with shadow batches, 66,245 total shadow batch entries):

flags value	Count	flags2 value	Count
0x10	56,698	0x02	42,358
0x90	7,216	0x03	14,511
0x80	1,406	0x00	5,778
0x00	421	0x01	3,415
0xC0	149	0x05	72
0x18	102	0x07	48

12.5 HD Model Identification

HD models can be identified via the M2 global flags. The strongest discriminator is **bit 20** (`0x00100000`):

Flag bit	HD models	Non-HD models	Notes
bit 20 (<code>0x00100000</code>)	72.0%	2.2%	Strongest HD indicator
bit 4 (<code>0x00000010</code>)	0.0%	76.4%	Never in HD
bit 12 (<code>0x00001000</code>)	0.0%	47.6%	Never in HD
bit 21 (<code>0x00200000</code>)	88.0%	82.0%	Not discriminating

No single bit is a perfect binary classifier. HD models typically carry flag `0x00302080` (bit 7 + bit 13 + bit 20 + bit 21), while non-HD models commonly carry `0x00202080` or `0x00202090`.

13. Chunked M2 (MD21 + Chunks)

13.1 Chunk Format

```
struct Chunk {
    char tag[4];    // ASCII, NOT byte-reversed (unlike other WoW formats)
    u32  size;      // payload size in bytes (excludes the 8-byte header)
    u8   data[size];
};
```

Chunks can appear in any order. WhiteoutLib's parser handles them order-independently via a dispatch loop. The writer emits them in a fixed deterministic order.

13.2 Chunk Catalogue

WhiteoutLib implements 30 chunk types. The following catalogue includes corpus statistics from 9059 M2 files:

Chunk	Since	Description	Corpus Files	Size Range
MD21	Legion	Core model data (MD20 payload)	9059	512–38 MB
PFID	Legion	Physics .phys fileDataId	—	4
SFID	Legion	Skin + LOD skin fileDataIds	9059	4–32
AFID	Legion	Animation .anim fileDataIds	109	8–592
BFID	Legion	Bone .bone fileDataIds	26	20–120
TXAC	Legion	Texture combiner parameters	4631	2–474
EXPT	Legion	Extended particle data v1	55	12–120
EXP2	Legion 7.3	Extended particle data v2	3138	48–14 KB
PABC	Legion 7.3	Parent sequence blacklist	1	32
PADC	Legion 7.3	Parent texture weights	9	21–49 KB
PSBC	Legion 7.3	Parent sequence bounds	25	2.5–11 KB
PEDC	Legion 7.3	Parent event data	25	29–108 KB
SKID	Legion 7.3	Skeleton .skel fileDataId	61	4
TXID	BfA	Texture fileDataIds (replaces filenames)	9019	4–220
LDV1	BfA	LOD configuration (16 bytes)	3630	16
RPID	BfA 8.1	Recursive particle model fileDataIds	4	8–24
GPID	BfA 8.1	Geometry particle model fileDataIds	12	8–84
WV1	BfA 8.2	Waterfall/PBR marker v1	—	0

Chunk	Since	Description	Corpus Files	Size Range
WV2	BfA 8.2	Waterfall/PBR marker v2	—	0
PGD1	Classic 1.13	Particle geoset data (M2Array-style)	2077	32–480
WV3	SL	Waterfall/PBR data v3	—	~96
PFDC	SL	Inline physics data	19	352–5584
EDGF	SL	Edge fade parameters (24 B/entry)	685	32–624
NERF	SL	Distance-based alpha (16 bytes)	1	16
DETL	SL	Detail light parameters (12 B/entry)	278	16–64
DBOC	SL	Distance-based opacity control (16 B/entry)	1022	16–512
AFRA	DF	Animation frame rate	—	—
PCOL	11.1.7	Player housing collision mesh	14	384–11 KB
DPIV	11.1.7	Pivot displacement data (32 B/entry)	15	32–64
TEXTL	12.0.0	Textured light parameters (16 B/entry)	3	16

Corpus note: No unknown chunk tags were found in the 9059-file corpus. All chunks belong to the 30 documented types above. WV1, WV2, WV3, AFRA, and PFID were not observed (the corpus may not include waterfall models or physics-heavy models).

13.3 File ID Reference Chunks

These chunks replace filename-based file discovery with integer file data IDs (CASC storage):

PFID — Physics:

```
struct PFIDChunk {
    u32 physFileDataId;
};
```

SFID — Skins:

```
struct SFIDChunk {
    std::vector<u32> skinFileDataIds;    // count = numSkinProfiles
    std::vector<u32> lodSkinFileDataIds; // remaining entries
};
```

Some models (e.g., `NightborneFemaleCitizen.m2`) have fewer bytes in this chunk than expected — 4 skin files + 2 LOD files but only 20 bytes (4 skins + 1 LOD).

AFID — Animations:

```

struct AFIDEntry {
    u16 animId;
    u16 subAnimId;
    u32 fileDataId;    // 0 = none
};
struct AFIDChunk {
    std::vector<AFIDEntry> entries;
};

```

BFID — Bones:

```

struct BFIDChunk {
    std::vector<u32> boneFileDataIds;
};

```

SKID — Skeleton:

```

struct SKIDChunk {
    u32 skeletonFileDataId;
};

```

TXID — Textures (replaces in-file filenames):

```

struct TXIDEntry {
    u32 fileDataId;
};
struct TXIDChunk {
    std::vector<TXIDEntry> entries;
};

```

RPID / GPID — Replace **M2ParticleOld** string filenames with fileDataIds:

```

struct M2RPIDChunk { std::vector<M2RPIDEntry> entries; }; // fileDataId per
particle
struct GPIDChunk    { std::vector<GPIDEntry>    entries; };

```

13.4 Particle Extension Chunks

TXAC — Texture/particle combiner hints:

```

struct TXACChunk {
    std::vector<std::array<u8, 2>> entries;
};

```

```
// count = materials.count + particles.count
};
```

Likely used in `CM2SceneRender::SetupTextureTransforms`. Internally influences `CParticleEmitter2` vertex buffer format selection.

EXPT — Extended particle v1 (possibly superseded by EXP2):

```
struct EXPTEntry {
    f32 zSource;
    f32 colorMult;
    f32 alphaMult;
};
```

If EXP2 doesn't exist, the client reconstructs it from EXPT data.

EXP2 — Extended particle v2:

```
struct EXP2Particle {
    f32 zSource;
    f32 colorMult;           // applied against particle's diffuse color
    f32 alphaMult;          // applied against particle's opacity
    // M2PartTrack<fixed16> alphaCutoff – per-particle alpha test over lifetime
};
```

The `alphaCutoff` track is indexed by the particle's current lifetime position. `colorMult` multiplies the particle's diffuse color, `alphaMult` multiplies opacity.

PGD1 — Particle geoset assignment:

```
struct PGD1Chunk {
    // M2Array-style header:
    u32 count;           // number of geoset entries (= particle emitter count)
    u32 dataOffset;      // byte offset from chunk data start to geosets array
                        // (always 16 in observed files)
    u32 reserved[2];     // always 0

    // At byte `dataOffset`:
    u16 geosets[count];  // per-emitter geoset ID (0 = always visible)
    // Padding to 16-byte alignment
};
// Total size = 16 + ceil(count * 2 / 16) * 16
// Minimum observed size = 32 bytes (for 1-8 particles)
```

The chunk uses an inline `M2Array<u16>`-style layout (count + offset at bytes 0–7) with the geoset data at the declared offset. Each `u16` value assigns the particle emitter to a geoset group, obeying the same geoset

visibility rules as `SkinSection.skinSectionId`. A value of `0` means the emitter is always visible. In the corpus (2077 files), most geoset entries are `0`; sparse non-zero values (e.g., 602, 206, 306) correspond to specific geoset IDs used for particle visibility gating.

13.5 Parent Data Chunks

Used when a model has a parent skeleton and needs to override or supplement parent animation data:

PABC — Parent sequence blacklist (called "BlacklistAnimData" internally):

```
struct PABCChunk {
    std::vector<u16> animationIds;
};
```

A flat array of animation IDs. If a target animation is found in this array, the parent's sequence lookup is bypassed. Otherwise, `parentSequencesLookups` are used.

PADC — Parent texture weights (replacement for `header.texture_weights`):

```
struct PADCCChunk {
    std::vector<TextureWeight> textureWeights;
};
```

PSBC — Parent sequence bounds:

```
struct PSBCChunk {
    std::vector<SequenceBounds> bounds; // Extent per parent sequence
};
```

PEDC — Parent event data:

```
struct PEDCChunk {
    std::vector<AnimationTrackBase> eventTracks;
};
```

13.6 LOD and Rendering Chunks

LDV1 — LOD configuration:

```
struct LDV1Chunk {
    u16 unknown0;
    u16 lodCount; // maxLod = lodCount - 1
    f32 unknown2; // used in: fmaxf(fminf(740.0/unk2, 5.0),
```

```
0.5)
    std::array<u8, 4> particleBoneLod; // per-LOD particle bone mask
    u32 unknown4;
};                                     // 16 bytes (confirmed: all 3630 files = 16B)
```

LOD skins (e.g., `_lod01.skin`) are selected based on `entityLodDist` and `doodadLodDist` CVars. The `particleBoneLod` array determines particle visibility at each LOD:

```
// For LOD level L:
u32 mask = 0x10000 << particleBoneLod[L];
// For each particle emitter:
if (mask & bones[particle.boneIndex].flags) {
    // Suppress this emitter at this LOD
}
```

EDGF — Edge fade:

```
struct EDGFEntry {
    f32 fadeStart;        // [0.0 .. 1.0] range – edge fade start distance factor
    f32 fadeEnd;          // [0.0 .. 1.0] range – edge fade end distance factor
    f32 fadeDistance;     // [0.0 .. 10.0] range – distance multiplier or falloff
    u32 submeshIndex;     // submesh/batch index reference (observed range 0–25)
    u32 reserved0;        // always 0
    u32 reserved1;        // always 0
};                        // 24 bytes per entry
// Chunk is padded to 16-byte alignment: size = ceil(count * 24 / 16) * 16
// e.g., 1 entry → 32B, 2 → 48B, 3 → 80B
```

Applied to meshes when `ShadowBatch.flags2 & 0x8` is set. 685 files in the corpus contain EDGF data.

NERF — Distance-based model alpha:

```
struct NERFChunk {
    f32 squaredFarDist;   // squared distance at which alpha = 0 (fade-out
complete)
    f32 squaredNearDist;  // squared distance at which alpha = 1 (fully visible)
    u32 reserved[2];      // always 0
};                        // 16 bytes total
// alpha = (squaredFarDist - squaredRadius) / (squaredFarDist - squaredNearDist)
```

This value multiplies the model instance's alpha, creating a fade-out effect based on distance. Only 1 file observed in corpus (values: 400.0 = 20² yards, 25.0 = 5² yards).

DETL — Detail light parameters:

```

struct DETLEntry {
    u16 flags;                // always 0 in corpus
    f16 scale;                // shadow RT matrix scale (corpus: constant
0x231C ≈ 0.00062)
    f16 diffuseColorMultiplier; // multiplier for M2Light.diffuse_color (corpus:
constant 0x3C00 = 1.0)
    u16 unk0;                 // always 0 in corpus
    u32 unk1;                 // always 0 in corpus
};                            // 12 bytes per entry
// count = header.lights.count
// Chunk padded to 16-byte alignment: 1 light → 16B, 2 lights → 32B

```

278 files in the corpus contain DETL data. All observed entries have identical constant values across the entire corpus (115 entries, 100 sampled files).

13.7 Waterfall / PBR Chunks

These chunks signal that the model uses a separate "waterfall" render path with PBR-ish rendering and normal maps. Despite the name, this technology is used for many non-waterfall models (Shadowlands environment dressing, etc.).

WV1 / **WV2** — Marker chunks. Empty payload structures in WhiteoutLib.

WV3 — Waterfall data v3:

```

struct WV3Data {
    f32 bumpScale;           // passed to vertex shader
    f32 value0_x, value0_y, value0_z;
    f32 value1_w, value0_w;
    f32 value1_x, value1_y;
    f32 value2_w, value3_y, value3_x;
    u8  baseColor[4];        // RGBA (not BGRA!)
    u16 flags;
    u16 unk0;
    f32 values3_w, values3_z, values4_y;
    f32 unk1, unk2, unk3, unk4;
};

```

The "value" fields are passed directly to the fragment shader.

13.8 Collision and Housing Chunks

PCOL — Player housing collision mesh (11.1.7+):

```

struct PCOLChunk {
    // Header:
    i32 vertexPosCount, vertexPosOffset;
    i32 faceNormCount, faceNormOffset;
};

```

```

    i32 indexCount, indexOffset;
    i32 flagsCount, flagsOffset;
    // Data (at respective offsets from chunk start):
    Vector3f vertexPositions[vertexPosCount];
    Vector3f faceNormals[faceNormCount];
    u16      indices[indexCount];
    u16      flags[flagsCount];
};

```

Use the offsets (relative to chunk data start) to seek — extra padding bytes may exist between sections.

DPIV — Pivot displacement data:

```

struct DPIVEntry {
    f32 offsetX;      // small displacement, often -0.0 or tiny negative
    f32 offsetY;      // small displacement, usually 0
    f32 offsetZ;      // varies significantly (-4.0 to +3.0 range)
    u32 flags;         // 0 or 1
    u32 reserved[4];   // always 0
};                    // 32 bytes per entry

struct DPIVChunk {
    DPIVEntry entries[]; // typically 1, observed up to 2
};
// Total size: N × 32 bytes. 14 files have 32 bytes (1 entry), 1 file has 64 bytes
// (2 entries).

```

15 files in the corpus contain DPIV data.

13.9 Miscellaneous Chunks

DBOC — Distance-based opacity control (seen in SL+):

```

struct DBOCEntry {
    f32 distanceOrScale; // range [0.001..10.0]; most common value ≈ 1.333
                          // (0x3FAAAAAB)
    f32 multiplier;      // range [0.0..13.0]; most common values: 1.5, 1.0
    u32 submeshIndex;     // submesh/batch reference (observed range 0-25)
    u32 reserved;         // always 0
};                        // 16 bytes per entry
// count varies: 1-12 entries observed per file
// Chunk size = count × 16 (always 16-byte aligned naturally)

```

1022 files in the corpus contain DBOC data. Size range: 16–512 bytes (1–32 entries). The `submeshIndex` values form a sequential pattern and appear to track which submesh each opacity control applies to.

AFRA — Animation frame rate (DF+):

```
struct AFRACHunk {
    std::vector<u8> data;
};
```

TEXTL — Textured lights (12.0.0+):

```
struct TEXTLEntry {
    f32 scaleU;           // UV scale U (corpus: always 1.0)
    f32 scaleV;           // UV scale V (corpus: always 1.0)
    i32 textureLookup;    // index in TXID for light cookie, or -1 (0xFFFFFFFF) if
    none
    i32 unk2;             // always 0 in corpus
};
// count = header.lights.count
```

Only 3 files in the corpus contain TEXTL data. All observed entries are identical: {1.0, 1.0, -1, 0}.

14. Skeleton File (.skel)

Skeleton files allow bone/attachment/sequence data to be shared across models via a parent-child hierarchy. The file is always chunked. When a model references a .skel via the SKID chunk, the skeleton data overrides or supplements the M2 header's bone/attachment/sequence arrays.

```
struct SkeletonFile {
    std::optional<SKL1Chunk> skl1_chunk;
    std::optional<SKA1Chunk> ska1_chunk;
    std::optional<SKB1Chunk> skb1_chunk;
    std::optional<SKS1Chunk> sks1_chunk;
    std::optional<SKPDChunk> skpd_chunk;
    std::optional<AFIDChunk> afid_chunk; // forwarded anim IDs
    std::optional<BFIDChunk> bfid_chunk; // forwarded bone IDs
};
```

SKL1 — Skeleton header:

```
struct SKL1Chunk {
    u32 flags;           // 0x100 seen in Legion
    std::string name;
    std::array<u8, 4> reserved;
};
```

SKA1 — Skeleton attachments:

```
struct SKA1Chunk {
    std::vector<Attachment> attachments;
    std::vector<u16> attachmentLookupTable;
};
```

SKB1 — Skeleton bones:

```
struct SKB1Chunk {
    std::vector<Bone> bones;
    std::vector<u16> keyBoneLookup;
};
```

SKS1 — Skeleton sequences:

```
struct SKS1Chunk {
    std::vector<GlobalSequence> globalLoops;
    std::vector<Sequence> sequences;
    std::vector<u16> sequenceLookups;
    std::array<u8, 8> reserved;
};
```

SKPD — Parent skeleton reference:

```
struct SKPDChunk {
    std::array<u8, 8> reserved0;
    u32 parentSkeletonFileId;    // fileDataId of parent .skel
    std::array<u8, 4> reserved1;
};
```

15. Bone File (.bone)

Bone files contain per-bone data for face-pose systems. The file uses chunked framing.

```
struct BoneFile {
    BONEHeader header;                // u32 unk = 1 (version?)
    std::optional<BIDACHunk> bida_chunk;
    std::optional<BOMTChunk> bomt_chunk;
    u32 boneId;                      // from filename
};
```

BIDA — Bone ID array:

```
struct BIDACHunk {
    std::vector<u16> boneIds; // count = (number of FacePose(808) sequences - 1)
};
```

BOMT — Bone offset matrices:

```
struct BOMTChunk {
    std::vector<Matrix4x4> boneOffsetMatrices; // same count as BIDA
};
```

16. Animation File (.anim)

Animation files externalize keyframe data for low-priority sequences (e.g., emotes, one-shot animations) to allow lazy loading.

Pre-Legion (raw)

The file is a raw binary blob of timestamps and values referenced by the M2's animation tracks. The .anim file to load is determined by:

```
{model_filename_without_extension}{anim_id:04d}-{sub_anim_id:02d}.anim
```

A file is loaded if `(sequence.flags & 0x130) == 0`. Blizzard's exporter aligns blocks to 16-byte boundaries, but this is not required.

Legion+ (chunked)

If the M2's `ChunkedAnimFiles` flag (0x2000) is set, or the model has a .skel file, .anim files use chunked framing:

```
struct AnimProfile {
    std::optional<AFM2Chunk> afm2_chunk; // animation data
    std::optional<AFSAChunk> afsa_chunk; // attachment animation data
    std::optional<AFSBChunk> afsb_chunk; // bone animation data
    bool isChunked;
};

struct AnimFile {
    AnimProfile profile;
    u32 animId; // from filename
    u32 variant; // from filename
};
```

- AFM2** — Contains the same animation data as old `.anim` files. For `.skel`-based models, this is restricted to event data only.
- AFSA** — Attachment animation data. Only present when the model uses a `.skel` file.
- AFSB** — Bone animation data. Only present when the model uses a `.skel` file.
- Detection logic:** WhiteoutLib peeks the first 4 bytes. If they match `AFM2`, `AFSA`, or `AFSB`, the file is chunked; otherwise, the entire file is treated as raw animation data stored in `afm2_chunk.animationData`.

17. Physics File (`.phys`) and PFDC

Physics data defines ragdoll simulation: rigid bodies, collision shapes, and constraints. Standalone `.phys` files use the same internal chunked format as PFDC inline data. All chunks use **reversed 4-byte ASCII tags** (standard WoW format), so `PHYS` on disk is stored as bytes `53 59 48 50`.

17.1 File Structure Overview

A physics file is a sequence of named chunks, each with the standard `u32_reversed_tag + u32_size + u8[size]` framing. The top-level chunk ordering observed in the corpus:

```
PHYS → PHYT → [CAPS] → [PLYT] → SHP2 → BDY4 → {joint_data} → [REVJ|REV2] → JOIN → [null_sentinel]
```

The null sentinel (`tag=0x00000000`, `size=0`) appears at the end of some PFDC inline chunks (9 of 19 v6 files).

17.2 Version History

Version	Introduced	Body	Shape	Weld	Shoulder	Revolute	Notes
0	MoP 5.0.1.15464	BODY	SHAP	WELJ	—	—	
1	Legion 7.0.1.20773	BODY	SHAP	WELJ	—	—	Added PHYT
2	Legion 7.0.1.20979	BDY2	SHP2	WLJ2	—	—	Renamed chunks, added fields
3	Legion 7.0.3.21287	BDY3	SHP2	WLJ2	—	—	Added PLYT polytope shapes
4	Legion 7.0.3.21846	BDY4	SHP2	WLJ2	SHOJ	REVJ	
5	Legion 7.3.0.24500	BDY4	SHP2	WLJ2	SHOJ	REVJ	PLYT header changes
6	≥7.3, ≤9.0	BDY4	SHP2	WLJ3	SHJ2	REV2	Newer joint structs

Corpus version distribution (54 total: 35 standalone **.phys** + 19 inline PFDC):

Version	Standalone	PFDC
4	24	0
5	11	0
6	0	19

17.3 PHYS Header

```
struct PHYSHeader {  
    u16 version;    // 0-6  
};
```

Chunk size is always 2 bytes.

17.4 PHYT — Physics Type

```
struct PHYTEntry {  
    u32 phyt;    // default 0  
};
```

Chunk size is always 4 bytes. Observed values by version:

Version	Values
4	always 3
5	always 3
6	4 (12×), 0 (4×), 3 (3×)

Likely meaning: ragdoll complexity or physics simulation type (0=none/simple, 3=standard, 4=enhanced).

17.5 Body Structures

The corpus uses **BDY4** exclusively (all 429 bodies across 54 files).

```
struct BDY4Entry {    // 48 bytes  
    u16 type;          // 0=kinematic/static, 1=dynamic  
    u16 boneIndex;     // model bone index  
    Vector3f position; // body position in bone space  
    u16 shapeIndex;    // starting index into SHP2 array  
    u8 padding_b[2];  
    i32 shapesCount;   // number of shapes in this body  
    f32 mass;          // (was unk0) body mass factor: 0.0-3.0 (typical: 0.5,  
    0.75, 1.0)
```

```
f32 massScale;      // (was unk_1c) mass multiplier: 1.0 (394x) or 10.0 (34x)
f32 drag;           // drag coefficient: 0.0-10.0
f32 angularDamping; // (was unk1) angular velocity damping: 0.0-20.0
f32 linearDamping;  // (was unk_28) linear velocity damping: 0.01-0.9 (default
0.9)
u8 unk_2c[4];       // flags/metadata (see below)
};
```

Body type distribution: type=0 (kinematic): 99, type=1 (dynamic): 330 **Bodies per file:** range 2–25 (most commonly 8–10)

Field value distributions (corpus-validated):

Field	Values	Interpretation
mass	1.0 (141x), 0.75 (141x), 0.5 (127x), 0.25 (6x), 0.1 (2x)	Body mass relative weight
massScale	1.0 (394x), 10.0 (34x), 0.0 (1x)	Multiplier applied to mass
drag	0.0 (128x), 5.0 (119x), 4.0 (102x), 3.0 (40x), 1.0 (29x)	Air resistance
angularDamping	5.0 (190x), 0.0 (106x), 10.0 (93x), 4.0 (29x)	Spin slowdown
linearDamping	0.01 (174x), 0.1 (102x), 0.5 (97x), 0.9 (34x), 0.25 (17x)	Movement slowdown

unk_2c analysis: Only non-zero on type=0 (kinematic root) bodies:

- v4: 0x0000C000 (alliance) or 0x00008000 (horde) — bit 14–15 flags
- v5: 0x00008000 consistently
- v6: small integers (3, 4, 9, 15, 19, 22) — collision group index?

17.6 Shape Structures

The corpus uses **SHP2** exclusively (all 426 shapes).

```
enum ShapeType : i16 {
    Box = 0,      // → BOXS data
    Capsule = 1,  // → CAPS data
    Sphere = 2,   // → SPHS data
    Polytope = 3  // → PLYT data (version 3+)
};

struct SHP2Entry {    // 32 bytes
    i16 shapeType;     // ShapeType enum
    i16 shapeIndex;    // index into the corresponding shape data chunk
    u8 unk[4];         // always 0x00000000 in corpus
    f32 friction;      // 0.0-0.7 (most common: 0.6)
    f32 restitution;   // 0.0-1.0 (most common: 0.0)
    f32 density;       // mass per unit volume (20000 for capsules, varies for
polytopes)
    u32 unk_14;        // always 0 in corpus
    f32 unk_18;        // 1.0 (425x) or 0.0 (1x) – likely a scale factor
```

```
    u16 unk_1c;           // always 0 in corpus
    u16 padding_1e;       // uninitialized in v4 (contains memory garbage), zeroed
in v5+
};
```

Shape type distribution: Capsule=131, Polytope=295, Box=0, Sphere=0

Physics material values:

Property	Most common values
friction	0.6 (226×), 0.5 (120×), 0.7 (50×), 0.2 (29×)
restitution	0.0 (238×), 0.1 (153×), 0.5 (34×)
density	20000 (85×, all capsules), 100–29326 (polytopes vary)

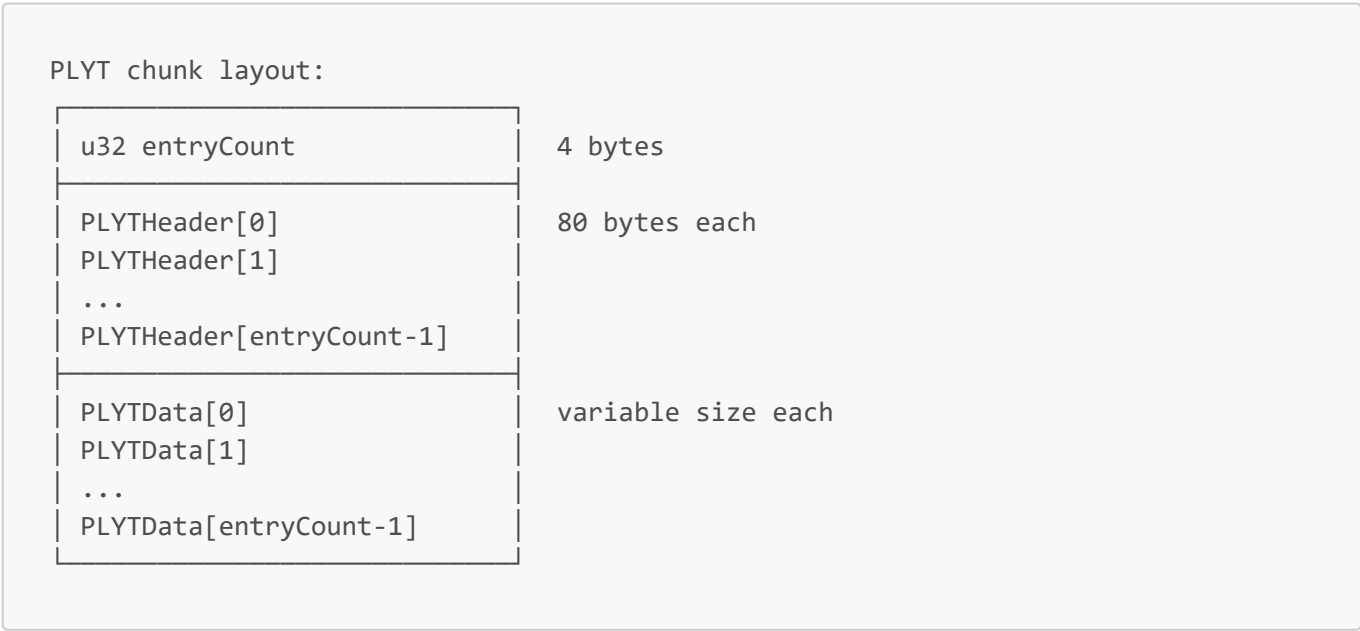
17.7 Shape Data: CAPS (Capsule)

```
struct CAPSEntry {    // 28 bytes
    Vector3f localPosition1;
    Vector3f localPosition2;
    f32 radius;
};
```

131 entries across 44 files. Chunk size = `entry_count × 28`.

17.8 Shape Data: PLYT (Polytope) — CORRECTED

The PLYT chunk layout differs significantly from the C++ struct definitions. The chunk uses a "headers-then-data" layout, NOT interleaved header+data per entry.



```

struct PLYTHeader {    // 80 bytes (0x50)
    u32 vertexCount;    // +0x00: convex hull vertex count
    u32 unk_04;         // +0x04: 0 in v4, non-zero in v5+ (version-specific
                        // metadata)
    u64 runtime_08_ptr; // +0x08: always 0 on disk, filled at runtime
    u32 faceCount;      // +0x10: number of faces (and face planes)
    u32 unk_14;         // +0x14: 1 in v4, varies in v5+ (version-specific)
    u64 runtime_18_ptr; // +0x18: always 0 on disk
    u64 runtime_20_ptr; // +0x20: always 0 on disk
    u32 nodeCount;      // +0x28: BSP tree node count
    u32 unk_2c;         // +0x2C: 0 in v4, non-zero in v5+
    u64 runtime_30_ptr; // +0x30: always 0 on disk
    f32 unk_38[6];      // +0x38: bounding data (pairs of Vector3f-like values)
};

struct PLYTData {
    Vector3f vertices[header.vertexCount];    // convex hull vertices
    Vector4f facePlanes[header.faceCount];    // face plane equations (nx, ny,
    nz, d)
    PLYTNode nodes[header.nodeCount];        // BSP tree for collision
    detection
    u8 faceIndices[header.faceCount];        // per-face index/metadata
};

struct PLYTNode {    // 4 bytes
    u8 byte0;        // node metadata / face reference
    u8 byte1;        // index value
    u8 byte2;        // child reference (0xFF = leaf)
    u8 byte3;        // child reference
};

```

Data size per entry: $\text{vertexCount} \times 12 + \text{faceCount} \times 16 + \text{nodeCount} \times 4 + \text{faceCount}$

Corpus statistics (all 295 PLYT entries are uniform):

- vertexCount = 8 (box-like convex hulls)
- faceCount = 6 (six-sided faces)
- nodeCount = 24
- entry data size = $8 \times 12 + 6 \times 16 + 24 \times 4 + 6 = 294$ bytes
- All runtime pointers are zero on disk ☒

Face planes are proper plane equations: $(0, 0, 1, 0.35)$, $(-1, 0, 0, 0.20)$, $(0, -1, 0, 0.02)$ etc.

Version-dependent header fields (unk_04, unk_14, unk_2c):

- v4: all zero (clean)
- v5: non-zero values consistent within a file (e.g., 110, 441, 648)
- v6: larger values, possibly runtime metadata or offsets

17.9 Joint Structure

```

struct JOINEntry {    // 16 bytes
    u32 bodyAIdx;      // index into body array
    u32 bodyBIdx;      // index into body array
    u8 unk[4];         // always 0x00000000 in corpus
    i16 jointType;      // JointType enum
    i16 jointId;       // joint sub-index within its type chunk
};

```

Joint type distribution: Shoulder=252, Weld=35, Revolute=62 **Joints per file:** range 1–18 (most commonly 6 or 9)

17.10 Joint Types

Version mapping:

- v4–5: SHOJ + WLJ2 + REVJ
- v6: SHJ2 + WLJ3 + REV2

SHOJ (Shoulder Joint) — 116 bytes

```

struct SHOJEntry {
    Matrix3x4 frameA;      // 48 bytes – reference frame in body A
    Matrix3x4 frameB;      // 48 bytes – reference frame in body B
    f32 lowerTwistAngle;    // -15.0 (112×) or -20.0 (84×) degrees
    f32 upperTwistAngle;    // 15.0 (112×) or 20.0 (86×) degrees
    f32 coneAngle;         // 45.0 (120×), 35.0 (60×), 20.0, 60.0 degrees
    f32 maxMotorTorque;     // 0.1 (157×), 1000.0 (35×), 0.01 (8×)
    u32 motorMode;         // 0=free (35×), 1=motor (165×)
};

```

SHJ2 (Shoulder Joint v2) — 124 bytes

Extends SHOJ with:

```

    f32 motorFrequencyHz;    // 1.0 (48×), 1.5 (2×), 3.0 (2×)
    f32 motorDampingRatio;   // 0.7 (39×) or 0.0 (13×)

```

WLJ2 (Weld Joint v2) — 112 bytes

```

struct WLJ2Entry {
    Matrix3x4 frameA;
    Matrix3x4 frameB;
    f32 angularFrequencyHz; // always 0.6 in corpus
    f32 angularDampingRatio; // always 0.0
    f32 linearFrequencyHz;   // always 0.0

```

```
f32 linearDampingRatio;    // always 0.0
};
```

WLJ3 (Weld Joint v3) — 116 bytes

Extends WLJ2 with:

```
f32 unk70;                // 0.0 (12×) or 0.005 (7×) – likely spring
stiffness
```

REVJ (Revolute Joint) — 112 bytes

```
struct REVJEntry {
    Matrix3x4 frameA;
    Matrix3x4 frameB;
    f32 lowerAngle;        // -60.0 (32×), -20.0, -30.0, -10.0 degrees
    f32 upperAngle;        // 60.0 (32×), 20.0, 30.0, 10.0 degrees
    f32 maxMotorTorque;     // 0.0 (16×), 0.1 (16×), 0.01 (5×)
    u32 motorMode;         // 0 (16×) or 1 (21×)
};
```

REV2 (Revolute Joint v2) — 120 bytes

Extends REVJ with:

```
f32 motorFrequencyHz;     // always 1.0 in corpus
f32 motorDampingRatio;    // always 0.7 in corpus
```

17.11 PFDC Inline Physics

When physics is inlined in the `.m2` via the **PFDC** chunk (Shadowlands+), the payload contains the same PHYS chunked structure. Some PFDC chunks end with a **null sentinel** (8 zero bytes: tag=0x00000000, size=0), observed in 9 of 19 v6 files.

All PFDC files in the corpus are version 6, using the latest struct variants (BDY4, SHP2, SHJ2, WLJ3, REV2).

WhiteoutLib currently stores PFDC as raw bytes: `m2file.pfdc_chunk->physicsData = reader.read<std::vector<u8>>(chunkSize)`

18. Bundle Naming Conventions

WhiteoutLib's `collectBundle()` discovers sibling files by stem-matching in the same directory as the `.m2`:

File Type	Pattern	Example
Base model	{stem}.m2	scorpion.m2
Skeleton	{stem}.skel	scorpion.skel
Base skin	{stem}{NN}.skin	scorpion00.skin, scorpion01.skin
LOD skin	{stem}_lod{NN}.skin	scorpion_lod01.skin
Animation	{stem}{NNNN}-{NN}.anim	scorpion0069-01.anim
Bone	{stem}_{NN}.bone	scorpion_07.bone

Where {NN} is zero-padded to 2 digits and {NNNN} to 4 digits.

`fromFileSystem()` performs the reverse: given an in-memory `FileSystem` and an output path, it computes all output file paths using the same conventions. Falls back to `"just_another_model"` if the base name is empty.

19. WhiteoutLib Implementation Notes

19.1 Parser

```
whiteout::m2::Parser parser(ParseMode::Lenient);
FileSystem fs = parser.parse("path/to/model.m2");
```

- **ParseMode::Strict**: throws on errors or unknown chunks
- **ParseMode::Lenient**: accumulates issues and continues; retrievable via `parser.getIssues()`

Parse order for bundles:

1. Base .m2 (detect MD20 vs MD21 by first 4 bytes)
2. Base skins (sorted by index)
3. LOD skins (sorted by LOD level)
4. .skel skeleton
5. .bone files (sorted by bone ID)
6. .anim files (sorted by anim ID, then variant)

Magic detection:

- 0x4D443230 ("MD20") → flat classic parser
- 0x4D443231 ("MD21") → chunked parser
- 0x534B494E ("SKIN") → skin parser

Offset resolution: The `BinaryParseVisitor` records `baseOffset` at the start of parsing. For `ClassicMD20`, `baseOffset = 0`. For `MD21`, `baseOffset` = position after the chunk header. All `M2Array` offsets resolve as `offset + baseOffset`.

Chunk parsing: Order-independent `while(hasRemaining())` loop with `switch(tag)`. After each handler, the reader seeks to `chunkStart + chunkSize` to guarantee forward progress even on partial reads.

Unknown chunks: Reported via the issue system and skipped by their declared size.

19.2 Writer

```
whiteout::m2::Writer writer;
writer.write("path/to/output.m2", fs);
```

- Writes base `.m2`, all skins, and `.skel` (if present) to disk
- `Writer::write(const BaseFile&)` returns `std::vector<u8>` for in-memory serialization
- Pre-allocates 2 MB buffer, then `shrink_to_fit()`

Chunk write order (deterministic): `MD21` → `LDV1` → `PFID` → `SFID` → `AFID` → `BFID` → `TXAC` → `EXPT` → `EXP2` → `PABC` → `PADC` → `PSBC` → `PEDC` → `SKID` → `TXID` → `RPID` → `GPID` → `WV1` → `WV2` → `PGD1` → `WV3` → `PFDC` → `EDGF` → `NERF` → `DETL` → `DBOC` → `AFRA` → `PCOL` → `DPIV` → `TEXL`

Chunks are written back-to-back with no inter-chunk alignment padding. Each chunk writes: tag (4B) → size placeholder (4B) → payload → seek back to patch actual size → return to end.

PFDC: Written directly as raw bytes, bypassing the structured writer visitor.

19.3 API Surface

Public headers:

Header	Purpose
<code>include/whiteout/m2/m2.h</code>	Umbrella include
<code>include/whiteout/m2/types.h</code>	Core types, version constants, tags
<code>include/whiteout/m2/structures.h</code>	Top-level FileSystem, BaseFile, SkinFile, etc.
<code>include/whiteout/m2/structures/base.h</code>	MD20Header and all contained structures
<code>include/whiteout/m2/structures/skin.h</code>	Skin profile, sections, batches
<code>include/whiteout/m2/structures/chunks.h</code>	All 30 chunk type definitions
<code>include/whiteout/m2/structures/skeleton.h</code>	Skeleton chunks
<code>include/whiteout/m2/structures/bone.h</code>	Bone file structures
<code>include/whiteout/m2/structures/anim.h</code>	Animation file structures
<code>include/whiteout/m2/structures/phys.h</code>	Physics structures
<code>include/whiteout/m2/parser.h</code>	Parser class
<code>include/whiteout/m2/writer.h</code>	Writer class

Top-level in-memory model:

```
struct FileSystem {
    std::string baseName;
    BaseFile base;
    std::vector<SkinFile> skins;
    std::vector<AnimFile> anims;
    std::vector<BoneFile> bones;
    std::optional<SkeletonFile> skeleton;
    std::optional<PhysicsFile> physics;
};
```

Typical usage:

```
#include <whiteout/m2/m2.h>

// Parse
whiteout::m2::Parser parser(whiteout::m2::ParseMode::Lenient);
whiteout::m2::FileSystem model = parser.parse("creature/scorpion/scorpion.m2");

// Access issues
for (const auto& issue : parser.getIssues()) {
    std::cerr << issue << "\n";
}

// Inspect header
auto& hdr = model.base.header;
std::cout << "Version: " << hdr.version << "\n";
std::cout << "Bones: " << hdr.bones.size() << "\n";
std::cout << "Vertices: " << hdr.vertices.size() << "\n";
std::cout << "Sequences: " << hdr.sequences.size() << "\n";

// Round-trip write
whiteout::m2::Writer writer;
writer.write("output/scorpion.m2", model);
```

Example programs:

- examples/m2_loader_example.cpp — Loads and prints model statistics
- examples/m2_writer_example.cpp — Loads, then writes to output path (round-trip)

19.4 Known Limitations

Area	Status
PFDC chunk	Stored and round-tripped as raw bytes; no structured parse
PFDC/PLYT parsing	phys.h PLYTEntry layout is wrong: file uses headers-then-data, not interleaved

Area	Status
AFRA chunk	Stored as raw <code>vector<u8></code>
Pre-WotLK skins	Inline skin profiles not implemented (skins must be external <code>.skin</code> files)
Pre-WotLK tracks	Single-timeline animation blocks (with <code>interpolation_ranges</code>) not implemented
Bone/anim file writing	<code>Writer::write(path, model)</code> does not emit <code>.bone</code> or <code>.anim</code> files; only base, skins, skeleton
Physics structured parse	<code>phys.h</code> defines all structures but the PFDC chunk does not use them
Vanilla rotation format	Full <code>C4Quaternion</code> (16-byte) bone rotation (pre-BC) not separately handled

20. Corpus Validation Results

This specification was validated against a corpus of **9059 M2 files** and **14443 SKIN files** from `Models/WoW/` (covering Dragonflight through The War Within 11.x era models). Analysis scripts are in `scripts/m2_corpus_analysis.py`, `scripts/m2_pgd1_analysis.py`, and `scripts/m2_seqflags_analysis.py`.

20.1 Overview

Statistic	Value
Total M2 files	9059
Classic (MD20)	0 (all files are chunked MD21)
Parse errors	0
SKIN files	14443
SKIN parse errors	0
Version 272 (1.16)	3629 files
Version 274 (1.18)	5430 files
Unique chunk tags	26 of 30 documented
Unknown chunk tags	0

20.2 Key Findings

Global Flags (Section 6.2):

- `0x80` is set on **all 9059 files** — universal in modern exports, not DH-tattoo-specific
- `0x10` appears in **5664 files** (62%) — previously undocumented, very common
- `0x200000` appears in **4274 files** (47%) — previously undocumented, very common
- `0x100` (CameraRelated) appears in only **3 files** — contradicts prior claim of "every model since Cata"

Sequence Flags (Section 7.3):

- 45310 total sequences across 9059 files (avg ~5/file)
- 10919 unique flag values
- Bit 0 (0x01) used in 19220 sequences — previously undocumented
- 0x7FFF (all bits 0–14) is the second-most common value (6347 sequences), typically for aliases
- Upper 16 bits non-zero in ~31% of sequences; when set, lower 16 bits are almost always 0xFFFF

Material Blend Modes (Section 8.4):

- Modes 5, 6, 7 discovered; mode 7 is **very common** (4133 materials)
- Mode 3 not observed in this corpus

Texture Types (Section 8.3):

- Types 21–24 not observed in the corpus
- Types 25 (4 files) and 26 (2 files) confirmed

PGD1 (Section 13.4):

- Completely re-documented: M2Array-style header (16 bytes) + u16 geoset data, not flat u16 per particle
- Size formula: $16 + \text{ceil}(\text{count} \times 2 / 16) \times 16$ — matches all 2077 files exactly

EDGF (Section 13.6):

- Entry size confirmed as 24 bytes with 3 identified fields (2×f32 fade factors, 1×f32 distance, 1×u32 index, 2×u32 reserved)

DBOC (Section 13.9):

- Entry size confirmed as 16 bytes: {f32 distance, f32 multiplier, u32 index, u32 reserved}
- 1022 files, 1–32 entries per file

SKIN Batches:

- Shadow batch flags distribution: 0x10 (56698), 0x90 (7216), 0x80 (1406)
- Shadow batch flags2 distribution: 0x02 (42358), 0x03 (14511), 0x00 (5778)
- Regular batch flags: 0x10 (62561), 0x80 (22942), 0x88 (10715)

20.3 Size Validation

Chunk	Validation	Result
TXID	size = textures.count × 4	☑ All match
SFID	size = numSkinProfiles × 4 (min)	☑ All match
PGD1	size = 16 + align16(particles × 2)	☑ All 2077 files match
LDV1	always 16 bytes	☑ All 3630 files match
EDGF	size = align16(entries × 24)	☑ All sizes explained

Chunk	Validation	Result
DBOC	size = entries × 16	☑ All sizes explained
DETL	size = align16(lights × 12)	☑ All sizes explained

20.4 PHYS Corpus Validation

Validated against **35 standalone .phys files** and **19 inline PFDC chunks** from M2 files (54 total). Analysis scripts: [scripts/m2_phys_analysis.py](#), [scripts/m2_phys_deep_analysis.py](#).

Statistic	Value
Standalone .phys files	35 (v4: 24, v5: 11)
Inline PFDC chunks	19 (all v6)
Parse errors	0
Size accounting errors	0 (all bytes accounted for)
Unknown chunks	0 (null sentinels are end-of-data markers)
Total bodies	429 (all BDY4)
Total shapes	426 (all SHP2: 131 capsule, 295 polytope)
Total joints	349 (252 shoulder, 35 weld, 62 revolute)
Total PLYT entries	295 (all 8-vertex, 6-face, 24-node)

Key PHYS findings:

- **PLYT layout corrected:** headers-then-data format with u32 count prefix (not interleaved)
- **PLYT data includes Vector4f face planes** (not u8 arrays as previously assumed)
- **BDY4 field names identified:** `unk0` → `mass`, `unk1` → `angularDamping`, `unk_28` → `linearDamping`
- **SHP2 `unk_1e`** is uninitialized padding (ASCII garbage in v4, zeroed in v5+)
- **Joint version correlation:** v4–5 use SHOJ/WLJ2/REVJ; v6 uses SHJ2/WLJ3/REV2
- **PFDC null sentinel:** 9 of 19 v6 PFDC chunks end with 8 zero bytes

21. References

1. **wowdev.wiki M2:** <https://wowdev.wiki/M2> — Community reverse-engineering documentation
2. **wowdev.wiki M2/.skin:** <https://wowdev.wiki/M2/.skin> — Skin file format
3. **wowdev.wiki M2/.skel:** <https://wowdev.wiki/M2/.skel> — Skeleton file format
4. **wowdev.wiki PHYS:** <https://wowdev.wiki/PHYS> — Physics file format
5. **wowdev.wiki BONE:** <https://wowdev.wiki/BONE> — Bone file format
6. **wowdev.wiki M2/Rendering:** <https://wowdev.wiki/M2/Rendering> — Shader and blend mode details
7. **Warcraft III Art Tools:** <http://ftp.blizzard.com/pub/war3/other/WarcraftIIIArtTools1.01.zip> — Historical MDX/PRE2 particle flag reference